

Software Engineering Lab — Complete Solutions

Lab 3: Component Modelling & Architectural Pattern Selection

SRN: PES1UG2AM293

NAME: SUMUKH D HEGDE

Problem Statement #39 — Retail, E-Commerce & Finance: **Second-Hand Product Escrow Marketplace**

Lab 3 Scenario: **Self-Service Coffee Kiosk System**

Lab 3

Component Modelling & Architectural Pattern Selection — Self-Service Coffee Kiosk

Scenario Recap

A Self-Service Coffee Kiosk in a busy café must let a customer choose from 3 coffee types (Espresso, Americano, Latte) and 2 sizes (Small, Large), pay by credit card only, and receive a printed receipt — all through a touch-screen interface connected to a physical receipt printer, with menu and pricing data stored locally.

1. Architectural Style Analysis

Style	Structure	Pros	Cons	Fit for this Kiosk
Layered	Horizontal layers: Presentation, Business, Data	Clear separation of concerns; easy to understand; good for traditional single-application systems	Some performance overhead from layer traversal; layers can become tightly coupled if boundaries are not enforced	Strong fit — the kiosk is one standalone application with a simple, linear order→pay→print workflow
Microservices	Small, independently deployable services	Independent scaling; fault isolation; technology diversity; independent deployment	Operational complexity; network latency; data-consistency challenges	Poor fit — a single physical kiosk has no need for independent scaling, and network hops would add latency to a device that must respond instantly to touch input

Client-Server	Centralized server with multiple clients	Centralized control; simple deployment; easy data consistency	Single point of failure; scalability bottleneck; limited fault tolerance	Partial fit — only the external card-authorization step is genuinely client-server; making the whole kiosk depend on a remote server risks the kiosk going fully offline if connectivity drops
----------------------	--	---	--	---

2. Written Justification (Architecture Selection)

We chose Layered Architecture for the Self-Service Coffee Kiosk System.

Reason 1 — Workflow matches the layers naturally

The kiosk has a well-defined, linear order → payment → receipt workflow. This maps cleanly onto a Presentation layer (touch-screen interaction), a Business layer (order orchestration and payment processing) and a Data/Hardware layer (menu & pricing storage, receipt printer). Keeping UI logic separate from payment logic makes the system far easier to test and maintain than an equivalent flat, unstructured implementation.

Reason 2 — No independent-scaling requirement

The kiosk is a single physical device with fixed, co-located hardware (touch-screen, card reader, printer). It does not need the independent deployment or independent scaling that Microservices exist to provide, so the operational overhead of running/orchestrating multiple services would be unjustified cost for no real benefit here.

Security Advantage

Confining all card-handling logic to the Payment Service component means the Presentation (UI) layer never touches raw card data directly — it only ever talks to the Order Manager through the `IOrderSubmission` interface. This shrinks the attack/PCI-DSS exposure surface: even if the touch-screen UI layer were compromised, an attacker could not directly reach payment internals, since the Payment Service is the single hardened chokepoint for all card authorization traffic to the external Card Payment Gateway.

Performance Benefit

The Data layer (Menu & Pricing Repository) can hold the small, rarely-changing menu and price list in memory rather than re-querying storage for every screen refresh. Because the Business layer only calls out to the external Card Payment Gateway once per transaction (at checkout) instead of repeatedly, order-to-payment latency stays low even during café rush hours when several customers may be queued at the kiosk.

3. UML Component Diagram

Six components are modelled (five required as a minimum): the two given components — **Order Manager** and **Payment Service** — plus **Touchscreen UI Controller** (Presentation), **Menu & Pricing Repository** and **Receipt Printer Adapter** (Data/Hardware), and the external **Card Payment Gateway**. Five provided/required interface pairs are shown using ball-and-socket notation, including the given Order Manager ↔ Payment Service interface, plus one «use» dependency (UI Controller directly reads live menu data for display).

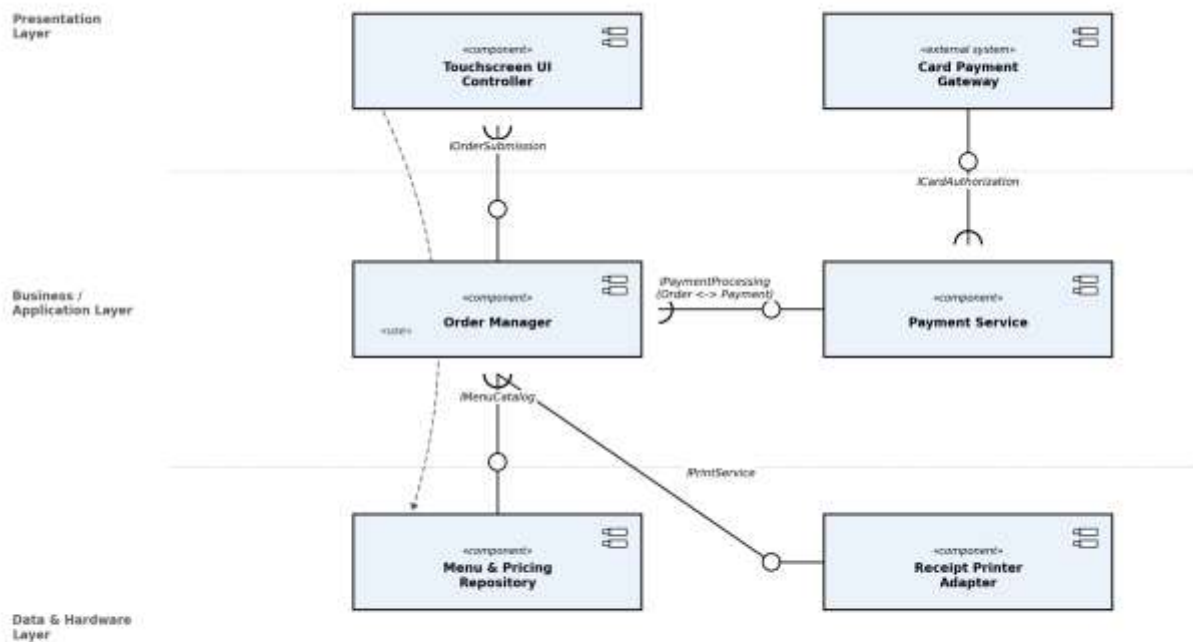


Figure 2. Component diagram — Self-Service Coffee Kiosk System

Component & Interface Summary

Component	Layer	Responsibility	Provided Interface	Required Interface
Touchscreen UI Controller	Presentation	Renders menu, captures coffee type/size selection, drives the order flow on screen	—	IOrderSubmission; «use» IMenuCatalog
Order Manager	Business	Orchestrates order creation and state; coordinates payment, pricing and printing	IOrderSubmission	IPaymentProcessing; IMenuCatalog; IPrintService
Payment Service	Business	Processes card payment requests for an order	IPaymentProcessing	ICardAuthorization
Menu & Pricing Repository	Data	Stores coffee types, sizes and current prices	IMenuCatalog	—
Receipt Printer Adapter	Hardware Interface	Drives the physical receipt printer	IPrintService	—

Component	Layer	Responsibility	Provided Interface	Required Interface
-----------	-------	----------------	--------------------	--------------------

Card Payment Gateway	External System	Authorizes and settles credit-card transactions	ICardAuthorization	—
----------------------	-----------------	---	--------------------	---